

# The Foundation of React

React is a frontend **JavaScript library** for building interactive **user interfaces** using components.

- **Created by Facebook (2011):** Initially built for Facebook's newsfeed and Instagram.
- **Open Source (2013):** First public release.
- **Adoption:** Now widely used by major companies like **Facebook, Instagram, Netflix, Reddit, Dropbox, Airbnb**, and many more.

## 1. Why it's called React

- React *reacts* to changes in data and updates the UI efficiently.
- You don't need to manually update the page—React does it automatically.

Example: when count changes, React re-renders only the `<p>` part, not the whole page.

## 2. What a Virtual DOM does

- The **Virtual DOM** is like a **copy** of the real DOM kept in memory.
- React compares the Virtual DOM with the previous version (“diffing”) and updates only what's changed.

// Imperative DOM update (Vanilla JS)

```
document.getElementById("title").innerText = "Hello World";
```

```
// Declarative update (React)

function App() {

  return <h1>Hello World</h1>;

}
```

React figures out *what exactly changed* and updates only that in the real DOM.

### 3. Composition vs. Inheritance

- Instead of **inheritance** (extending classes), React uses **composition** (combining components).

**Example: The Card component is reused to “wrap” other UI parts.**

## 4. Declarative vs. Imperative Programming

- **Declarative (React way):** You describe the UI for a given state.
- **Imperative (traditional JS):** You manually tell the DOM how to change step by step.

### Imperative Example:

```
const btn = document.createElement("button");
```

```
btn.innerText = "Click Me";
```

```
btn.onclick = () => alert("Clicked!");
```

```
document.body.appendChild(btn);
```

## Declarative Example (React):

```
function App() {  
  return <button onClick={() => alert("Clicked!")}>Click Me</button>;  
}
```

In React, you just **declare** the UI, and React handles the DOM updates.

## 5. Idiomatic React

- Writing code in the “**React way**” means following its principles:
  - Use **components**.
  - Use **state and props** for data flow.
  - Keep data **unidirectional** (parent → child).

## Why “React”?

- Facebook built React to handle **frequent UI updates** efficiently.

Web applications need to update in response to **events** such as:

- User input (typing, clicking, scrolling)
- New data from APIs or servers (e.g., live chat messages, news feed updates)
- Sensor data (e.g., GPS location changes)

Traditionally, web apps used a **refresh or polling model**:

- The page refreshes every few seconds to check for new data (*pull* model).

React introduced a **reactive model**:

- Instead of refreshing blindly, the UI **reacts** immediately to actual data changes (*push* model).

This style of updating UIs based on data changes is called **Reactive Programming**.

# 1. User Input

Whenever a user **types, clicks, or scrolls**, the website needs to react immediately.

- Example: When you type in a search box, results show up instantly.
- Example: When you click a “Like” button, it turns blue without refreshing the whole page.

React updates only the part of the page that changed (the search results or the button), not the entire website.

## 2. New Data from APIs or Servers

Websites often need to **fetch new data** from the internet.

- Example: In WhatsApp Web, new chat messages appear automatically.
- Example: On Facebook/Instagram, the news feed updates when new posts arrive.

React makes sure the new data is displayed instantly without reloading the whole page.

### 3. Sensor Data (like GPS)

Some apps depend on data from your device's **sensors**.

- Example: Google Maps updates your location in real time when you move.
- Example: A weather app changes to show weather for your current city.

React reacts to this incoming sensor data and updates only what's needed on the screen.

In short:

- **User input** → "I did something" → Update the UI.
- **API/server data** → "New info came in" → Update the UI.
- **Sensor data** → "Device data changed" → Update the UI.

## React and the UI Layer

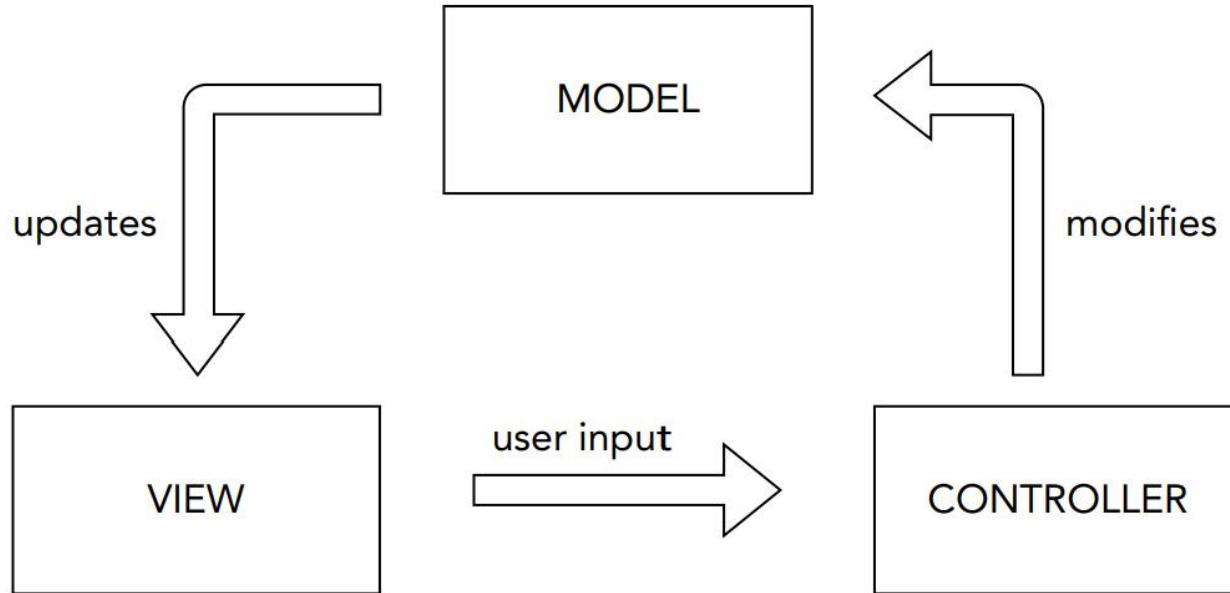
Web applications are often structured using the **MVC (Model–View–Controller) pattern**:

- **Model** → Data layer
- **View** → What the user sees (UI)
- **Controller** → Connects View ↔ Model

**React only cares about the “View”.**

- It takes data (state/props) as input.
- It renders that data as UI.

In an MVC application, the View sends input to the Controller, which passes data between the data layer and the View. React is only concerned with the V in MVC. It takes data as input and presents it to the user in some form.



**FIGURE 2-1:** The MVC pattern

React itself doesn't care **whether the user is using a mobile phone, a tablet, a desktop web browser, a screen reader, a command-line interface, or any other kind of device or interface that may be invented in the future.**

React just renders **components**. How those components get presented to the user is up to a separate library.

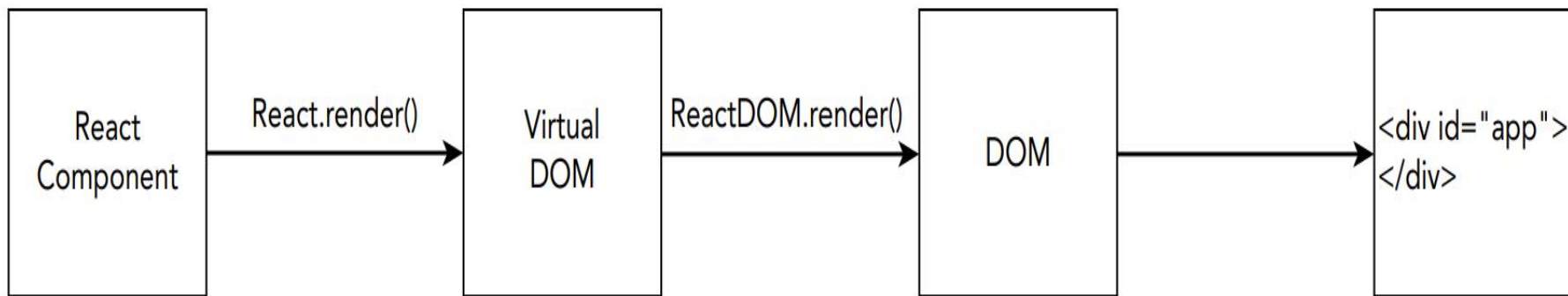
The library that handles rendering of React components in web browsers is called **ReactDOM**. If you want to render React elements to native mobile apps, you use **React Native**. If you want to render React components to static HTML, you can use **ReactDOMServer**.

## What is ReactDOM?

- React by itself just describes **what the UI should look like**.
- But React doesn't know *where* to put that UI — it needs help to connect with the **actual browser DOM**.
- That's the job of **ReactDOM**.

Think of React as the **architect** (designing components) and ReactDOM as the **builder** (actually putting them into the house = browser).

**ReactDOM** has a number of functions for interfacing between React and web browsers, but the one that every React application makes use of is called **ReactDOM.render**



**FIGURE 2-2:** React and ReactDOM

## 1. React Component

- You write UI as **React components** (functions or classes).
- **Example:**

```
function App() {  
  return <h1>Hello, React!</h1>;  
}
```

## 2. React.render() → **Virtual DOM**

- React takes the component and creates a **Virtual DOM** (a lightweight, in-memory copy of the actual DOM).
- This Virtual DOM is just a **JavaScript object representation** of what the UI should look like.
- Example (conceptually, not exact code):

```
{  
  type: "h1",  
  props: { children: "Hello, React!" }  
}
```

### 3. ReactDOM.render() → DOM

- ReactDOM compares the **Virtual DOM** with the real DOM.
- It then updates only the parts of the **real DOM** that need changes.
- This makes updates **fast and efficient** (instead of reloading the whole page).

### 4. DOM → `<div id="app"></div>`

- Finally, ReactDOM renders the component into the actual HTML page.
- Typically, you have an empty `<div id="app"></div>` (or "root") in your HTML file.
- React fills this div with your components.

**Example:**

```
<body>
```

```
  <div id="app"></div>
```

```
  <script src="bundle.js"></script>
```

```
</body>
```

After **ReactDOM.render(<App />, document.getElementById("app"));**, the  
<div> will now contain:

```
<div id="app">
```

```
  <h1>Hello, React!</h1>
```

```
</div>
```

## In Short

- **React Component** → Written by developer.
- **Virtual DOM** → Internal React representation.
- **ReactDOM.render()** → Syncs Virtual DOM with Real DOM.
- **Real DOM** → Browser updates only what changed.
- **Result** → Efficient, dynamic UI inside `<div id="app">`.

## Example

```
function Counter() {  
  const [count, setCount] = React.useState(0);  
  return (  
    <div>  
      <h1>Count: {count}</h1>  
      <button onClick={() => setCount(count + 1)}>Increase</button>  
    </div>  
  );  
}
```

## React Creates a Virtual DOM

```
{  
  type: "div",  
  children: [  
    { type: "h1", children: "Count: 0" },  
    { type: "button", children: "Increase" }  
  ]  
}
```

You click **Increase**.

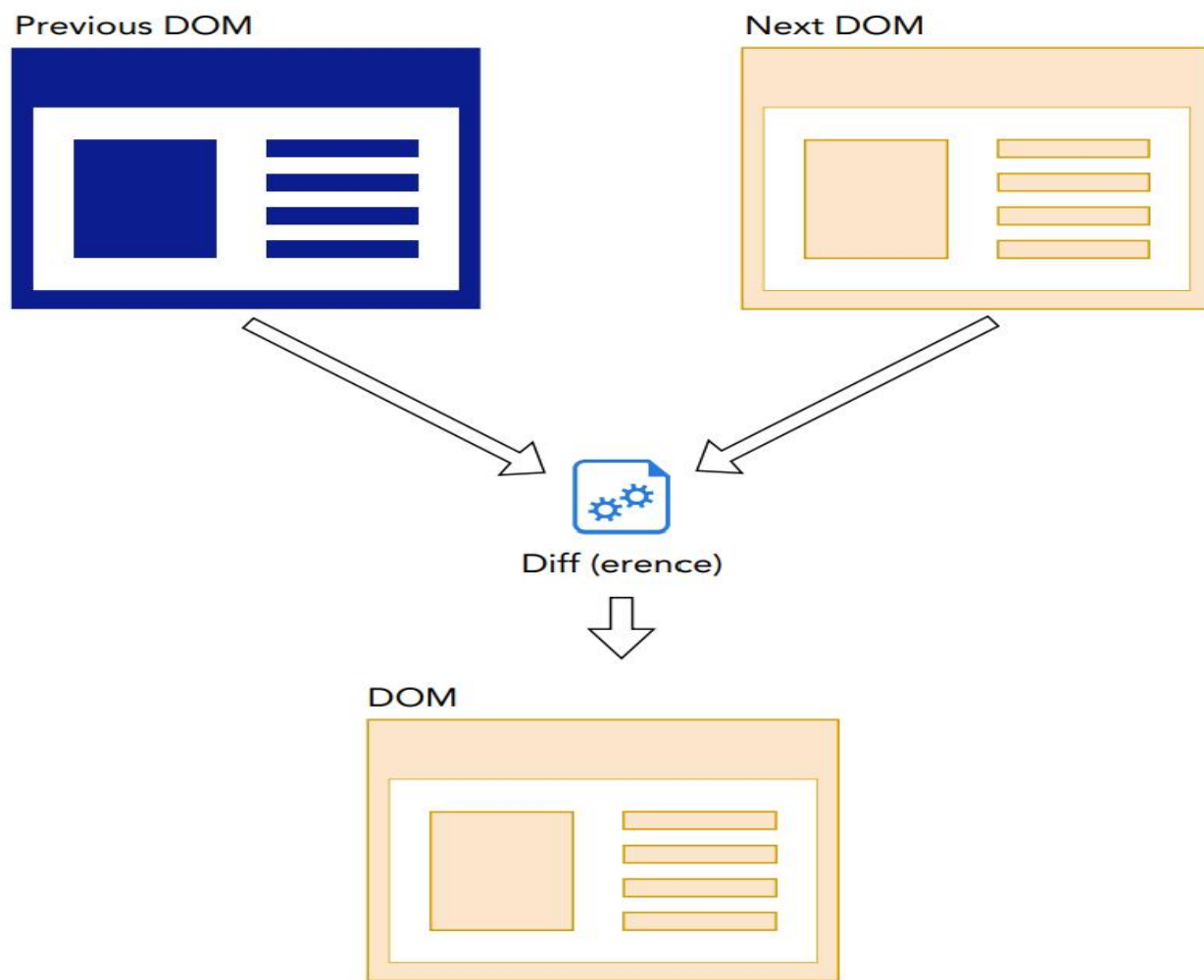
The count state changes from  $0 \rightarrow 1$ .

React **re-renders** the Virtual DOM with the new data.

```
{  
  type: "div",  
  children: [  
    { type: "h1", children: "Count: 1" }, // only this part changed  
    { type: "button", children: "Increase" }  
  ]  
}
```

## ReactDOM Updates the Real DOM

- React compares the **old Virtual DOM** with the **new Virtual DOM**.
- It finds that only the text in `<h1>` changed (Count: 0 → Count: 1).
- ReactDOM updates **just that part** in the real DOM.



**FIGURE 2-3:** How the Virtual DOM works

# THE PHILOSOPHY OF REACT

## 1. Thinking in Components

- React apps are built from **components** (small reusable pieces of UI).
- Each component should have **one responsibility** (Single Responsibility Principle).

// A general Button component

```
function Button(props) {  
  return <button onClick={props.handleClick}>{props.label}</button>;  
}
```

// A Submit button (special case, using the same Button)

```
function SubmitButton() {  
  return <Button label="Submit" handleClick={() => alert("Form Submitted!")} />;  
}
```

- Button is a **reusable component**.
- It takes **props** (parameters) from outside:
  - props.label → what text should appear inside the button.
  - props.handleClick → what function should run when the button is clicked.
- **Example:**

```
<Button label="Cancel" handleClick={() => alert("Cancelled!")} />
```

This would create a button that says **Cancel** and shows an alert when clicked.

## Submit Button Component

```
function SubmitButton() {  
  return <Button label="Submit" handleClick={() => alert("Form Submitted!")} />;  
}
```

Here, instead of writing a **new button from scratch**, we reuse the Button component.

We **pass specific props**:

- `label="Submit"` → text on the button is **Submit**.
- `handleClick={() => alert("Form Submitted!")}` → when clicked, it shows “Form Submitted!”

Without React, you'd probably write a new `<button>` every time with its own logic.

With React, you make **one general Button** → then reuse it by passing **different props**.

This follows the **composition philosophy** → build specific components by reusing general ones.

# Composition vs. Inheritance

In **Object-Oriented Programming (OOP)**, we often use **inheritance**.

Example in pseudo-code (Java / C++ style):

```
class Button {  
    void render() {  
        System.out.println("General Button");  
    }  
}
```

```
class SubmitButton extends Button {  
    void render() {  
        System.out.println("Submit  
Button");  
    }  
}
```

Imagine you are building a website with many types of buttons:

- Submit button
  - Cancel button
  - Login button
  - Logout button
- You have to make a **new child class** for every different type of button.
  - If you want to add one more (e.g., “Download Button”), you must create another class.
  - This becomes **too many files, too much repetition**, and is **hard to maintain**.

## The React Way (Composition + Props)

React avoids this problem. Instead of creating a **new child class every time**, you just make **one general Button** and customize it using **props**.

```
function Button(props) {  
  return <button onClick={props.handleClick}>{props.label}</button>;  
}
```

// Specific cases using the same Button

```
function App() {  
  return (  
    <div>  
      <Button label="Submit" handleClick={() => alert("Form Submitted!")} />  
      <Button label="Cancel" handleClick={() => alert("Cancelled!")} />  
      <Button label="Login" handleClick={() => alert("Logged In!")} />  
      <Button label="Logout" handleClick={() => alert("Logged Out!")} />  
    </div>  
  );  
}
```

- Only **one Button component** exists.
- Different types (Submit, Cancel, Login, Logout) are created by **changing props**.
- No duplicate classes. No huge inheritance tree. Easy to maintain.

# React Is Declarative

## Imperative Programming (Step by Step Instructions)

In imperative programming, you tell the computer **exactly how to do something**, step by step.

### Example with jQuery:

```
<p id="para1">Old text</p>
```

```
<script>
```

```
  // Imperative approach
```

```
  $('#para1').html('<p>This is the new paragraph.</p>');
```

```
</script>
```

## Explanation:

- You tell the browser:
  1. Go find the element with id="para1".
  2. Replace its inner HTML with new text.
- You (the programmer) are in charge of **how** the DOM changes.

This is like telling a robot:

- Walk to the bread.
- Open the bag.
- Take out two slices.
- Find a knife... etc.

## Declarative Programming (Describe the End Result)

In declarative programming, you just **describe what you want**, not how to do it. The system (React in this case) figures out the steps.

Example with **React**:

```
function App() {  
  return <p>This is the new paragraph.</p>;  
}
```

## Explanation:

- You don't manually find the paragraph or replace text.
- You just say: **"The UI should look like this."**
- React compares old UI vs new UI using the **Virtual DOM** and updates only what's changed.

This is like telling a robot:

- "Make me a peanut butter sandwich."  
The robot already knows the steps, so you don't have to explain them.

| Approach         | Imperative (Before React)                  | Declarative (React)                      |
|------------------|--------------------------------------------|------------------------------------------|
| Programmer's Job | Tell <b>how</b> to update DOM step by step | Just say <b>what UI should look like</b> |
| Example          | <code>\$('#para1').html("new text")</code> | <code>&lt;p&gt;new text&lt;/p&gt;</code> |
| Maintenance      | Hard (manual updates everywhere)           | Easy (UI auto-syncs with state)          |

# React Is Idiomatic

React is **just JavaScript** with a small set of extra tools.

If you already know JavaScript, React won't feel alien — it uses familiar patterns.

The more **JavaScript features** you learn (functions, objects, ES6+, arrow functions, destructuring, classes), the better you get at React.

# Javascript

```
function greet(name) {  
  return "Hello, " + name;  
}  
  
console.log(greet("Anitha")); // Hello, Anitha
```

## **React component (same idea, just returns JSX):**

```
function Greet(props) {  
  return <h1>Hello, {props.name}</h1>;  
}
```

Both are **functions**.

- One returns a **string** ("Hello, Shruti") → normal JS.
- One returns **JSX** (<h1>Hello, Shruti</h1>) → React.

So, React components are **just functions in JavaScript**.

That's why it's called **idiomatic** → looks natural if you already know JS.

# React vs Angular

## 1. Library vs Framework

- **React:** Just a **library** (a tool) for building UI with components.  
Like Lego blocks — you choose what other tools you need.
- **Angular:** A **framework** (full system) with everything built-in (routing, forms, HTTP, etc.).  
Like a full Lego kit — you get everything, but it's heavier and has rules.

## 2. Learning Curve

- **React:** Easier to start (just JavaScript + JSX).
- **Angular:** Harder to start → you must also learn **TypeScript** and Angular's rules.

## 3. DOM Handling

- **React:** Uses **Virtual DOM** (makes updates fast).
- **Angular:** Uses **Real DOM + Change Detection** (updates directly but can be slower).

## With Angular (Real DOM + Change Detection):

Angular makes this automatic using **two-way data binding** (`[(ngModel)]`).

```
<input [(ngModel)]="username" placeholder="Type your name">
```

```
<p>Hello {{username}}</p>
```

And in Angular's component code:

```
export class AppComponent {  
  username: string = "";  
}
```

1. Angular watches (detects changes) in the username variable.
2. If the **model changes** (username = "Anitha"), Angular updates the `<p>` automatically.
3. If the **user types** in the input, Angular updates the variable username.

This is called **Change Detection**. Angular checks the **real DOM** for changes and keeps the UI + data in sync.

## 4. Data Flow

- **React: One-way data flow**

- Data goes **one direction**: from parent → child.

- Example:

```
function Child({ message }) {  
  return <p>{message}</p>;  
}
```

```
function Parent() {  
  return <Child message="Hello from Parent!" />;  
}
```

Child gets data from Parent. If Parent changes, Child updates.  
But Child **cannot directly change Parent's data.**

- **Angular: Two-way data binding**

- Data flows **both ways**: View ↔ Model.

**Example (Angular style with [(ngModel)]):**

```
<input [(ngModel)]="username">
```

```
<p>Hello {{username}}</p>
```

- If user types in the input, the username variable changes.  
If the code changes username, the input updates automatically.

## **React (one-way data flow):**

Like giving instructions to your assistant:

- *You tell them what to do → they do it.*
- If you change your mind, you give new instructions.
- The assistant never changes your mind on their own.

## **Angular (two-way binding):**

Like a walkie-talkie conversation:

- *You talk, and they reply at the same time.*
- Both sides can change the situation directly.

| Feature     | React (Library)              | Angular (Framework)          |
|-------------|------------------------------|------------------------------|
| Type        | Library (small, flexible)    | Full framework (all-in-one)  |
| Learning    | Easier (JS, JSX)             | Harder (TypeScript + rules)  |
| DOM         | Virtual DOM (faster)         | Real DOM + Change Detection  |
| Data Flow   | One-way (Parent → Child)     | Two-way (View ↔ Model)       |
| Flexibility | High (choose your own tools) | Low (you follow Angular way) |

## Similarities (React & Vue)

- Both are used to build **modern, dynamic user interfaces**.
- Both use **components** (small reusable UI pieces).
- Both support **props (data passing)** and **state (data storage)**.
- Both are **fast**, thanks to Virtual DOM.

| Feature        | React                                         | Vue                                                         |
|----------------|-----------------------------------------------|-------------------------------------------------------------|
| Type           | Library (focused only on UI)                  | Framework (UI + more features built-in)                     |
| Language       | Pure JavaScript + JSX (HTML inside JS)        | HTML templates + JS (looks closer to plain HTML)            |
| Data Flow      | One-way data flow (parent → child)            | Two-way data binding possible (v-model)                     |
| Learning Curve | Medium (need JSX + state management)          | Easier (looks like regular HTML/JS/CSS)                     |
| Ecosystem      | Needs external libraries (like Router, Redux) | Vue has built-in tools (Vue Router, <u>Vuex</u> )           |
| Adoption       | Backed by Facebook/Meta, huge community       | Backed by Evan You + community, very popular in Asia/Europe |

## What React Is Not

React is a **front-end library for building user interfaces**, running entirely in the browser. It **does not replace servers, databases, or development tools**, and it is **not a one-size-fits-all solution**.

## 1. React Is Not a Web Server

- React cannot handle HTTP requests or serve web pages.
- A web server sends React (and other JS/CSS/HTML files) to the browser, but React itself cannot act as the server.
- React only runs **in the browser** and interacts with it via JavaScript APIs.

## Example

Suppose you visit **www.example.com**.

The **web server** sends you:

- index.html (the structure)
- style.css (the styling)
- app.js (React library and your React code)

Your browser opens these files.

React runs in the browser and dynamically updates the page:

Shows a list of products

Changes the content when you click a button

But **React cannot serve new pages to other people on its own; that's the server's job.**

## React Is Not a Programming Language

- React is a **JavaScript library**, not a language.
- It's a collection of pre-written JavaScript functions to simplify creating dynamic user interfaces.
- You could, in theory, write React functionality yourself purely in JavaScript, but the library saves you time.

## React Is Not a Database Server

- React cannot store data permanently or securely.
- Dynamic data in a React UI (like session data) is temporary.
- Persistent data must be handled via server-side databases, APIs, or browser storage (cookies/local storage).

## React Is Not a Development Environment

- React does not require any specific tools or IDEs.
- You can write React code with any text editor or toolchain.
- Common tools (like Node.js, npm, or Vite) are optional; they just make development easier.
- React is just a library. You **can run it in a plain HTML file** or use **modern tools** to improve development speed and workflow.

## React Is Not the Perfect Solution for Everything

- React is great for many applications, but it's not ideal for all situations(Simple, Static Websites, Heavy Backend Processing Apps, Non-Web Applications,etc)
- Knowing other languages and libraries helps you choose the best tool for the job rather than defaulting to React.

## JSX is not HTML

```
function Login(){  
  const handleSubmit = (e) => {  
    e.preventDefault();  
    console.log(`logging in ${e.target[0].value}`);  
    // do something else here  
  }  
}
```

```
return (  
  <form id="login-form" onSubmit={handleSubmit}>  
    <input type="email" id="email" placeholder="E-Mail Address"/>  
    <input type="password" id="password"/>  
    <button>Login</button>  
  </form>  
);  
}  
export default Login;
```

The function Login() is a **React component**.

Inside the return, it looks like you're returning HTML, but it's actually **JSX**.

JSX allows you to **write XML-like syntax in JavaScript**, which React then **compiles into regular JavaScript** (calls to React.createElement) under the hood.

## What is JSX?

- **JSX** stands for **JavaScript XML**.
- It is an **XML-like syntax** that you can write inside **JavaScript**.
- It allows you to **describe what your UI should look like** in a **syntax that resembles HTML**.

## Why it's not HTML

Even though it looks like HTML:

- It's embedded in JavaScript, so you can't just paste raw HTML in a JS file.
- Every JSX element **must be closed**, even `<input />` or `<br />`.
- Some attribute names are different:
  - **class** → **className**
  - **for** → **htmlFor**
- You can embed **JavaScript expressions** inside JSX using `{}` braces.

**For example:**

```
const username = "React";
```

```
return <p>Hello, {username}!</p>;
```

This will render: **Hello, React!**

## The JSX you wrote:

```
<form id="login-form" onSubmit={handleSubmit}>
```

```
  <input type="email" id="email" placeholder="E-Mail Address"/>
```

```
  <input type="password" id="password"/>
```

```
  <button>Login</button>
```

```
</form>
```

**Gets compiled by React into something like this:**

```
React.createElement(  
  "form",  
  { id: "login-form", onSubmit: handleSubmit },  
  React.createElement("input", { type: "email", id: "email", placeholder: "E-Mail  
Address" }),  
  React.createElement("input", { type: "password", id: "password" }),  
  React.createElement("button", null, "Login")  
);
```

So, it **becomes pure JavaScript**, which the browser can execute.

JSX = JavaScript + HTML-like syntax.

It **cannot run directly as HTML**, but React transforms it into JavaScript that manipulates the DOM.

This makes **React components declarative and readable**, letting you visualize UI while staying in JavaScript.

# JSX Syntax

## JSX ≠ HTML

JSX **looks like HTML** but it's actually **JavaScript XML** — which means it has stricter rules than HTML.

## XML-Like Rules in JSX

### 1. All elements must be closed

`<br />` ✓ `<br>` ✗ Since JSX is JavaScript, **every tag needs to be properly closed**. This ensures that the code is syntactically correct when it's converted to JavaScript.

1. **Empty elements need a self-closing slash**- Certain HTML elements don't have children, and these are known as "empty elements." Examples include `<img>`, `<br>`, `<input>`, `<link>`, etc. In HTML, you might write them without a closing tag (e.g., `<br>`), but in JSX, **they must be self-closed** with a slash (/). ``

## Non-empty elements

a) `<div>`

`<h1>Welcome to my site</h1>`

`<p>This is a paragraph inside a div.</p>`

`</div>`

a) `<p>This is a paragraph.</p>`

b) `<h1>This is a main heading</h1>`

c) `<a href="https://www.example.com">Click here to visit Example</a>`

### 3. String attributes must be in quotes

`<input type="text" name="username" />`

### 4. HTML tags must be lowercase

`<div></div>` ✓ `<Div></Div>` ✗

### 5. Reserved HTML Attribute Names (due to JS conflicts)

`<label htmlFor="username">Username</label>`

**JS:**

`<label for="username">Username</label>`

| HTML  | JSX       |
|-------|-----------|
| class | className |
| for   | htmlFor   |

## 6. camelCase for Multi-Word Attributes

In JSX, multi-word HTML attributes are **camelCased**:

| HTML     | JSX      |
|----------|----------|
| onclick  | onClick  |
| tabindex | tabIndex |

## 7. Custom Attributes

### For Built-in DOM Elements

- Before React 16: Use data- prefix. **<div data-color="blue" data-size="large">My Item</div>**
- React 16+: Can use any attribute name (though still not best practice).

**<div color="blue" size="large">My Item</div>**

## For User-Defined Components

Custom attributes = **props** for components:

```
<MyWidget widgetSize="large" color="blue" />
```

## 8. Boolean Attributes

You can omit the value or pass true:

```
<input disabled />    // ✓
```

```
<input disabled={true} /> // ✓
```

## 9. Embedding JavaScript: Use {}

For values that are dynamic (like variables or functions), wrap in curly braces:

```
<input value={props.term} onChange={(e) => props.onChange(e.target.value)} />
```

## 10. Object Literals = Double Curly Braces

Use double braces for inline style objects:

```
<h1 style={{ fontSize: "24px", color: "blue" }}>Hello</h1>
```

The first pair of curly braces {} is for JSX to evaluate the JavaScript expression. The second pair {} is for the **object literal**.

## 11. Comments in JSX

Use JavaScript block comments, wrapped in {}:

```
{/* This is a comment */}
```

```
function MyComponent(props) {  
  return (  
    <div className="container">  
      {/* This is a header */}  
      <h1 style={{ fontSize: "20px", color: "green" }}>  
        Hello, {props.username}  
      </h1>  
    </div>  
  );  
}
```

```
<input
  type="text"
  name="search"
  value={props.searchTerm}
  onChange={(e) => props.onChange(e.target.value)}
  disabled
/>
</div>
);
}
```

**Install Node.js** (if you don't already have it). -

<https://nodejs.org/en/download>

```
npx create-react-app my-app
```

# OR (faster)

```
npm create vite@latest my-app
```

```
cd my-app
```

```
npm install
```



Learn

About

Download

Blog

Docs

Contribute ↗

Certification ↗

Q Start typing...

Ctrl + K



# Download Node.js®

Get Node.js®

v22.20.0 (LTS) ▾

for

 Windows ▾

using

 Docker ▾

with

 npm ▾

**Info** Want new features sooner? Get the [latest Node.js version](#) instead and try the latest improvements!

```
1 # Docker has specific installation instructions for each operating system.
2 # Please refer to the official documentation at https://docker.com/get-started/
3
4 # Pull the Node.js Docker image:
5 docker pull node:22-alpine
6
7 # Create a Node.js container and start a Shell session:
8 docker run -it --rm --entrypoint sh node:22-alpine
9
10 # Verify the Node.js version:
11 node -v # Should print "v22.20.0".
12
13 # Verify npm version:
14 npm -v # Should print "10.9.3".
```

PowerShell



Copy to clipboard

9 vulnerabilities (3 moderate, 6 high)

To address all issues (including breaking changes), run:

```
npm audit fix --force
```

Run `npm audit` for details.

Created git commit.

Success! Created my-app at C:\Users\MY PC\my-app

Inside that directory, you can run several commands:

`npm start`

Starts the development server.

`npm run build`

Bundles the app into static files for production.

`npm test`

Starts the test runner.

`npm run eject`

Removes this tool and copies build dependencies, configuration files and scripts into the app directory. If you do this, you can't go back!

We suggest that you begin by typing:

```
cd my-app
```

```
npm start
```

Happy hacking!

`npm notice`

`npm notice` New major version of npm available! 10.9.2 -> 11.6.1

`npm notice` Changelog: <https://github.com/npm/cli/releases/tag/v11.6.1>

`npm notice` To update run: `npm install -g npm@11.6.1`

`npm notice`

C:\Users\MY PC>|

```
C:\Users\MY PC>npx create-react-app my-app
```

```
Need to install the following packages:
```

```
create-react-app@5.1.0
```

```
Ok to proceed? (y) y
```

```
npm warn deprecated fstream-ignore@1.0.5: This package is no longer supported.
```

```
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache :  
a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerful.
```

```
npm warn deprecated uid-number@0.0.6: This package is no longer supported.
```

```
npm warn deprecated rimraf@2.7.1: Rimraf versions prior to v4 are no longer supported
```

```
npm warn deprecated glob@7.2.3: Glob versions prior to v9 are no longer supported
```

```
npm warn deprecated fstream@1.0.12: This package is no longer supported.
```

```
npm warn deprecated tar@2.2.2: This version of tar is no longer supported, and will not receive security updates. Please  
ap.
```

```
create-react-app is deprecated.
```

```
You can find a list of up-to-date React frameworks on react.dev
```

```
For more info see:https://react.dev/link/cra
```

```
This error message will only be shown once per install.
```

```
Creating a new React app in C:\Users\MY PC\my-app.
```

```
Installing packages. This might take a couple of minutes.
```

```
Installing react, react-dom, and react-scripts with cra-template...
```

```
||
```

MY PC

← → ↑ ↺

Local Disk (C:) > Users > MY PC >

Search MY PC

+ New



Sort View

...

De

Documents

Pictures

Music

Videos

Ayur\_vision\_doc

semicon

sumedha

LMS-prod\_new

> This PC

> Network

> Linux

| Name        | Date modified    | Type           |
|-------------|------------------|----------------|
| Downloads   | 24-09-2025 23:04 | File folder    |
| Favorites   | 16-06-2025 05:54 | File folder    |
| Links       | 16-06-2025 05:54 | File folder    |
| Music       | 16-06-2025 05:54 | File folder    |
| my-app      | 25-09-2025 11:53 | File folder    |
| OneDrive    | 27-07-2025 11:38 | File folder    |
| Pictures    | 03-08-2025 23:34 | File folder    |
| Postman     | 17-07-2025 22:40 | File folder    |
| Saved Games | 16-06-2025 05:54 | File folder    |
| Searches    | 16-06-2025 11:42 | File folder    |
| Videos      | 31-08-2025 20:50 | File folder    |
| .gitconfig  | 17-07-2025 23:25 | GITCONFIG File |
| .lessht     | 02-09-2025 10:58 | LESSHST File   |
| NTUSER.DAT  | 24-09-2025 23:05 | DAT File       |

my-app

←

→

↑

↺

Search my-app

+

New

✂

📄

📄

🔍

📁

🗑

↕

Sort

≡

View

⋮

📄

Data

📁 Documents

📁 Pictures

📁 Music

📁 Videos

📁 Ayur\_vision\_doc

📁 semicon

📁 sumedha

📁 LMS-prod\_new

> This PC

> Network

> Linux

| Name                | Date modified    | Type             |
|---------------------|------------------|------------------|
| 📁 .git              | 25-09-2025 11:53 | File folder      |
| 📁 node_modules      | 25-09-2025 11:59 | File folder      |
| 📁 public            | 25-09-2025 11:53 | File folder      |
| 📁 src               | 25-09-2025 11:53 | File folder      |
| 📄 .gitignore        | 25-09-2025 11:53 | txtfile          |
| 📄 package.json      | 25-09-2025 11:53 | JSON Source File |
| 📄 package-lock.json | 25-09-2025 11:53 | JSON Source File |
| 📄 README.md         | 25-09-2025 11:53 | MD File          |

|                   | Name                 | Date modified    | Type                  |
|-------------------|----------------------|------------------|-----------------------|
| 📁 Documents       | 📄 App.css            | 25-09-2025 11:53 | CSSfile               |
| 📁 Pictures        | 📄 App.js             | 25-09-2025 11:58 | JSFile                |
| 📁 Music           | 📄 App.test.js        | 25-09-2025 11:53 | JSFile                |
| 📁 Videos          | 📄 index.css          | 25-09-2025 11:53 | CSSfile               |
| 📁 Ayur_vision_doc | 📄 index.js           | 25-09-2025 11:53 | JSFile                |
| 📁 semicon         | 🖼️ logo.svg          | 25-09-2025 11:53 | Microsoft Edge HTM... |
| 📁 sumedha         | 📄 reportWebVitals.js | 25-09-2025 11:53 | JSFile                |
| 📁 LMS-prod_new    | 📄 setupTests.js      | 25-09-2025 11:53 | JSFile                |

- > 🖥️ This PC
- > 🌐 Network
- > 🐧 Linux

Restricted Mode is intended for safe code browsing. Trust this window to enable all features. [Manage](#) [Learn More](#)

JS App.js X Release Notes: 1.104.1

C: > Users > MY PC > my-app > src > JS App.js > MyComponent

```
1 function MyComponent(props) {  
3   <div className="container">  
4     /* This is a header */  
5     <h1 style={{ fontSize: "20px", color: "green" }}>  
6       Hello, {props.username}  
7     </h1>  
8     <input  
9       type="text"  
10      name="search"  
11      value={props.searchTerm}  
12      onChange={(e) => props.onChange(e.target.value)}  
13      disabled  
14    />  
15  </div>  
16  );  
17 }  
18  
19 export default function App() {  
20   return /
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\MY PC\my-app> npm start

You can now view my-app in the browser.

Local: http://localhost:3000  
On Your Network: http://172.30.160.1:3000

Note that the development build is not optimized.  
To create a production build, use `npm run build`.

webpack compiled **successfully**  
[]



React App



localhost:3000

# Hello, Shruti

Hello world

# What is Conditional Rendering?

It means:

“Show this **only if** some condition is true.”

Or: “Show **A** if condition is true, otherwise show **B**.”

In React, we often need this when showing different UI for **logged in** vs **logged out** users, error messages, loading screens, etc.

# 1. Conditional Rendering with if/else + element variables

```
import Header from './Header';  
  
import Login from './Login';  
  
function Welcome({ loggedIn }) {  
  let content;  
  
  if (loggedIn) {  
    content = <Header />;  
  
  } else {  
    content = <Login />;  
  
  }  
}
```

```
return (  
  <div>  
    {content}  
  </div>  
);  
  
}  
  
export default Welcome;
```

What happens:

- If `loggedIn = true` → `<Header />` is shown.
- If `loggedIn = false` → `<Login />` is shown.

```
function Welcome({ loggedIn }) {  
  let message;  
  if (loggedIn) {  
    message = <h2>Welcome back!</h2>;  
  } else {  
    message = <h2>Please log in</h2>;  
  }  
}
```

```
return (
```

```
  <div>
```

```
    {message}
```

```
  </div>
```

```
);
```

```
}
```

```
function App() {  
  let isLoggedIn = true;  
  if (isLoggedIn) {  
    return <h1>Welcome, User!</h1>;  
  }  
  else {  
    return <h1>Please Login</h1>;  
  }  
}  
  
export default App;
```

## 2. Conditional Rendering with && (logical AND)

```
import Header from './Header';

function Welcome({ loggedIn }) {

  return (

    <div>

      {loggedIn && <Header />}

    </div>

  );

}

export default Welcome;
```

How it works:

- `loggedIn && <Header />` means:
  - If `loggedIn = true` → show `<Header />`.
  - If `loggedIn = false` → show nothing.

### 3. Conditional Rendering with ? : (Ternary Operator)

```
import Header from './Header';  
  
import Login from './Login';  
  
function Welcome({ loggedIn }) {  
  return (  
    <div>  
      {loggedIn ? <Header /> : <Login />}  
    </div>  
  );  
}  
  
export default Welcome;
```

How it works:

- If `loggedIn = true` → show `<Header />`.
- If `loggedIn = false` → show `<Login />`.

```
function App() {
```

```
  const isLoggedIn = true; // Try changing this to false
```

```
  return (
```

```
    <div>
```

```
      {/* 1. if/else with variables */}
```

```
      {isLoggedIn ? <h2>Welcome back!</h2> : <h2>Please log in</h2>}
```

```
      {/* 2. && operator */}
```

```
      {isLoggedIn && <p>You have 5 new notifications</p>}
```

```
      {/* 3. Normal always-visible element */}
```

```
      <p>This text is always shown.</p>
```

```
    </div>
```

```
  );
```

```
}
```

```
export default App;
```



React App



localhost:3000

# Welcome back!

You have 5 new notifications

This text is always shown.

# Expressions in JSX

## What can go inside { }?

- Numbers: `{1+1}` → shows 2
- Strings: `{"Hello " + "World"}` → shows Hello World
- Variables: `{username}`
- Function calls: `{myFunction()}`
- Immediately invoked functions: `{(() => "value")()}`

## What cannot go directly inside { }?

- if statements
- for loops
- Function declarations

These don't produce a **value** directly, so you must run them *outside* JSX.

## Examples:

1. `<h1>{1 + 1}</h1>`

2. 

```
function App() {  
  return (  
    <div>  
      <h1>{1 + 1}</h1>  
      <h1>{10 * 5}</h1>  
      <h1>{100 / 4}</h1>  
    </div> );  
}  
  
export default App;
```

3. `<p>My age is {20 + 5}</p>`

4. `<h1>{"Hello " + "World"}</h1>`

```
import { useState } from "react";
```

```
function CountUp() {
```

```
  const [count, setCount] = useState(0);
```

```
  return (
```

```
    <div>
```

```
      {/* Button calls a function (expression) on click */}
```

```
      <button onClick={() => setCount(count + 1)}>Add One</button>
```

```
      {/* Expression: shows the value of count */}
```

```
      {count}
```

```
    </div>
```

```
  );
```

```
}
```

```
export default CountUp;
```

Here:

- `onClick={() => setCount(count+1)}` is an **arrow function** (expression)
- `{count}` is also an **expression** (value of the variable count).

## Example 2: Immediately Invoked Function

```
function ImmediateInvoke() {  
  return (  
    <div>  
      {(() => <h1>The Header</h1>())}  
    </div>  
  );  
}  
  
export default ImmediateInvoke;
```

Here:

Inside { }, we create a function:  
( ) => <h1>The Header</h1>

- Then **immediately call it** with ( ).
- So it returns <h1>The Header</h1> which JSX can render.

## Why would we use an IIFE in JSX?

- It can be useful when you need **several JavaScript statements or a small calculation** inside JSX.

```
function App() {  
  return (  
    <div>  
      {(() => {  
        let a = 10;  
        let b = 20;  
        let result = a + b;  
  
        return <h1>Result: {result}</h1>;  
      })()}  
    </div>  
  );  
}
```

```
export default App;
```

```
function App() {  
  let a = 10;  
  let b = 20;  
  let result = a + b;  
  
  return <h1>Result: {result}</h1>;  
}
```

## Using all five together:

```
function myFunction() {  
  return "Hello from function!";  
}  
  
function App() {  
  let username = "Anitha";  
  
  return (  
    <div>  
      <h1>Expressions in JSX</h1>  
  
      <p>Number: {1 + 1}</p>  
  
      <p>String: {"Hello " + "World"}</p>  
  
      <p>Variable: {username}</p>  
  
      <p>Function: {myFunction()}</p>  
  
      <p>IIFE: {(() => "value")}</p>  
    </div>  
  );  
}  
  
export default App;
```

## output:

Expressions in JSX

Number: 2

String: Hello World

Variable: Anitha

Function: Hello from function!

IIFE: value

# Using Children in JSX

## The rule

A React component's **return statement must return only one “thing”**.

- That “thing” can be:
  - A string: "Hello"
  - A number: 123
  - A Boolean: true or false
  - An array: [<A />, <B />]
  - A **single JSX element**

Even if you want to show **multiple elements**, they must be **inside a single parent element**.

## Example of invalid JSX

```
return(  
  <MyComponent />  
  <MyOtherComponent />  
);
```

This is **wrong** because there are **two sibling elements** at the top level. React doesn't know which one to return.

## Fix: Wrap with a parent element

return(

<div>

<MyComponent />

<MyOtherComponent />

</div>

);

Now it's valid JSX because:

- There is **one parent** **<div>** wrapping the children.
- React sees **one element** being returned, but inside it can have **many children**.

## Alternative: Using React Fragments

If you **don't want an extra <div>**, you can use **React Fragment**:

```
return(
```

```
  <>
```

```
    <MyComponent />
```

```
    <MyOtherComponent />
```

```
  </>
```

```
);
```

- `<> ... </>` is a **special tag that won't render extra HTML**.
- It still allows **multiple children** to be returned.

## What are React Fragments?

When you return **multiple JSX elements** from a component, React requires **one parent element**.

- Sometimes people wrap them in `<div>...</div>` just to satisfy this rule.
- But adding extra `<div>`s **creates unnecessary HTML**, which can make your HTML messy and affect styling or accessibility.

**React Fragment** solves this problem:

- It **wraps multiple elements** into a single JSX element
- **But doesn't create extra HTML** in the DOM

## How to use React.Fragment : Using <React.Fragment>

```
import React from "react";

function MyComponent() {
  return (
    <React.Fragment>
      <h1>The heading</h1>
      <h2>The subheading</h2>
    </React.Fragment>
  );
}

export default MyComponent;
```

- Here <h1> and <h2> are children of <React.Fragment>.
- **No extra <div> is added in the HTML.**

## Using Fragment from React

```
import { Fragment } from "react";

function MyComponent() {
  return (
    <Fragment>
      <h1>The heading</h1>
      <h2>The subheading</h2>
    </Fragment>
  );
}

export default MyComponent;
```

## Short syntax: <> </>

```
function MyComponent() {  
  return (  
    <>  
      <h1>The heading</h1>  
      <h2>The subheading</h2>  
    </>  
  );  
}  
  
export default MyComponent;
```

- Shortest and cleanest way.
- **No import needed.**
- Works exactly the same as <Fragment>

# What is JSX Transform?

JSX is **not valid JavaScript** on its own. Browsers cannot understand it.

The **JSX transform** is the **process of converting JSX into plain JavaScript** that browsers can run.

## ◇ Example

JSX code:

```
const element = <h1 className="title">Hello, React!</h1>;
```

After **JSX transform** (by Babel):

```
const element = React.createElement(  
  "h1",  
  { className: "title" },  
  "Hello, React!"  
);
```

- `<h1>` → `"h1"` (type)
- `className="title"` → `{ className: "title" }` (props)
- `"Hello, React!"` → child of the element

This is **plain JavaScript**, which the browser can execute.

## Why JSX Transform is needed

1. JSX looks like HTML but is **not JavaScript**.
2. React needs **React.createElement** calls to build elements.
3. Babel or another compiler **transforms JSX** → **JS** automatically.

## What is Babel?

- Babel is a **JavaScript compiler**.
- It allows you to write **modern JavaScript and JSX** and converts it into **browser-compatible JavaScript**.
- Browsers **cannot understand JSX** directly. Babel turns JSX into plain JavaScript.

## Why we need Babel with React

React code often looks like this:

```
function App() {  
  return <h1>Hello, World!</h1>;  
}
```

- `<h1>Hello, World!</h1>` is **JSX**, not valid JavaScript.
- Browsers don't understand JSX.
- Babel compiles it to this:

```
function App() {  
  return React.createElement("h1", null, "Hello, World!");  
}
```

- Now the browser can run it because it's **plain JS**.

## Example with JSX and Babel

### Input (JSX):

```
const element = <h1>Hello React!</h1>;
```

### Compiled by Babel (JavaScript):

```
const element = React.createElement("h1", null, "Hello React!");
```

- `React.createElement` → React's way to create DOM elements.
- Every JSX tag is converted to a `React.createElement` call.